Git & GitHub Basics

1- Why version control matters

2- Git vs GitHub (they're different!)

3- Setting up Git & GitHub accounts

4- Basic Git commands (add, commit, push)

From chaos to control!

GitHub

SCROLL

The Problems with Manual Versioning

Manual Versioning = Nightmare

Why This is a Nightmare:

- Can't see what changed between versions
- Can't easily go back to a previous version
- Can't collaborate without overwriting each other's files
- Don't know which file is actually current
- No record of WHY changes were made
- Massive file clutter
- Can't easily work across multiple computers



There has to be a better way!

The Solution – Version Control

Version Control = Time Machine for Code

Git tracks EVERY change to your files:

- See complete history of every edit ever made
- Undo anything — infinite undo!
- Collaborate without overwriting each other
- Label important versions
- Never lose work — even if your laptop dies
- Document WHY changes were made
- Work from any computer, always in sync



Git remembers everything so you don't have to.

Git turns your project folder into a complete, searchable history of every decision you ever made.

Why Git Matters Professionally

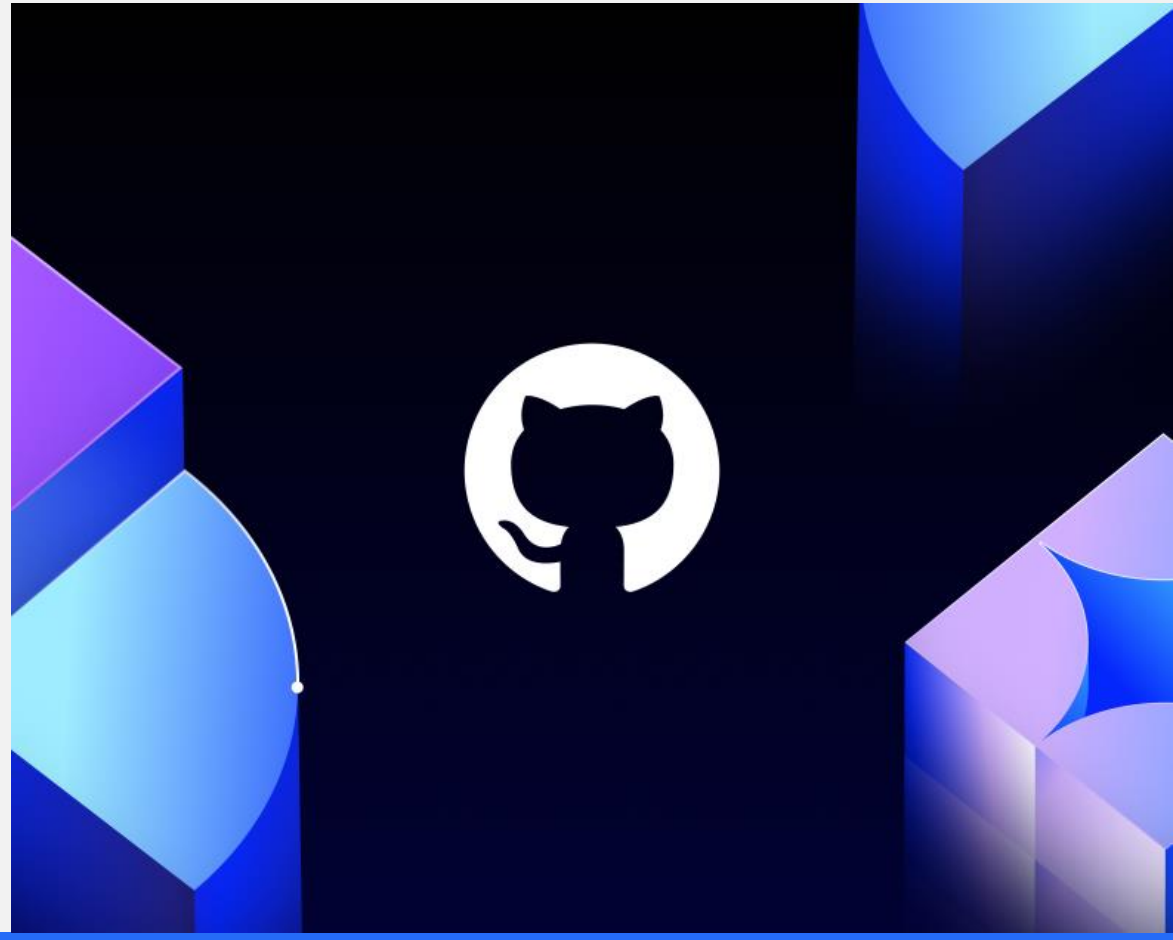
Industry Standard — No Exceptions:

Every tech company uses Git:

- Google ✓
- Microsoft ✓ (they own GitHub!)
- Facebook/Meta ✓
- Apple ✓
- Netflix ✓
- Every startup ✓

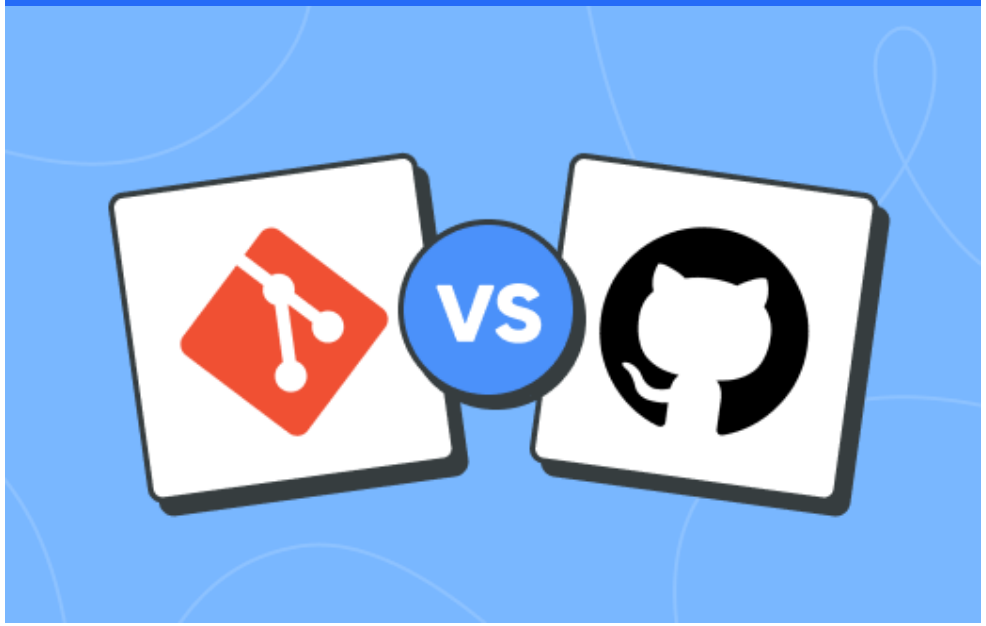
Git appears in over 90% of web developer job postings.

Your GitHub profile is your coding resume — employers scroll through it.



Git vs GitHub – What's the Difference?

Git is the tool you work with. **GitHub** is the place you share from. Knowing the difference is the first step to working like a professional.



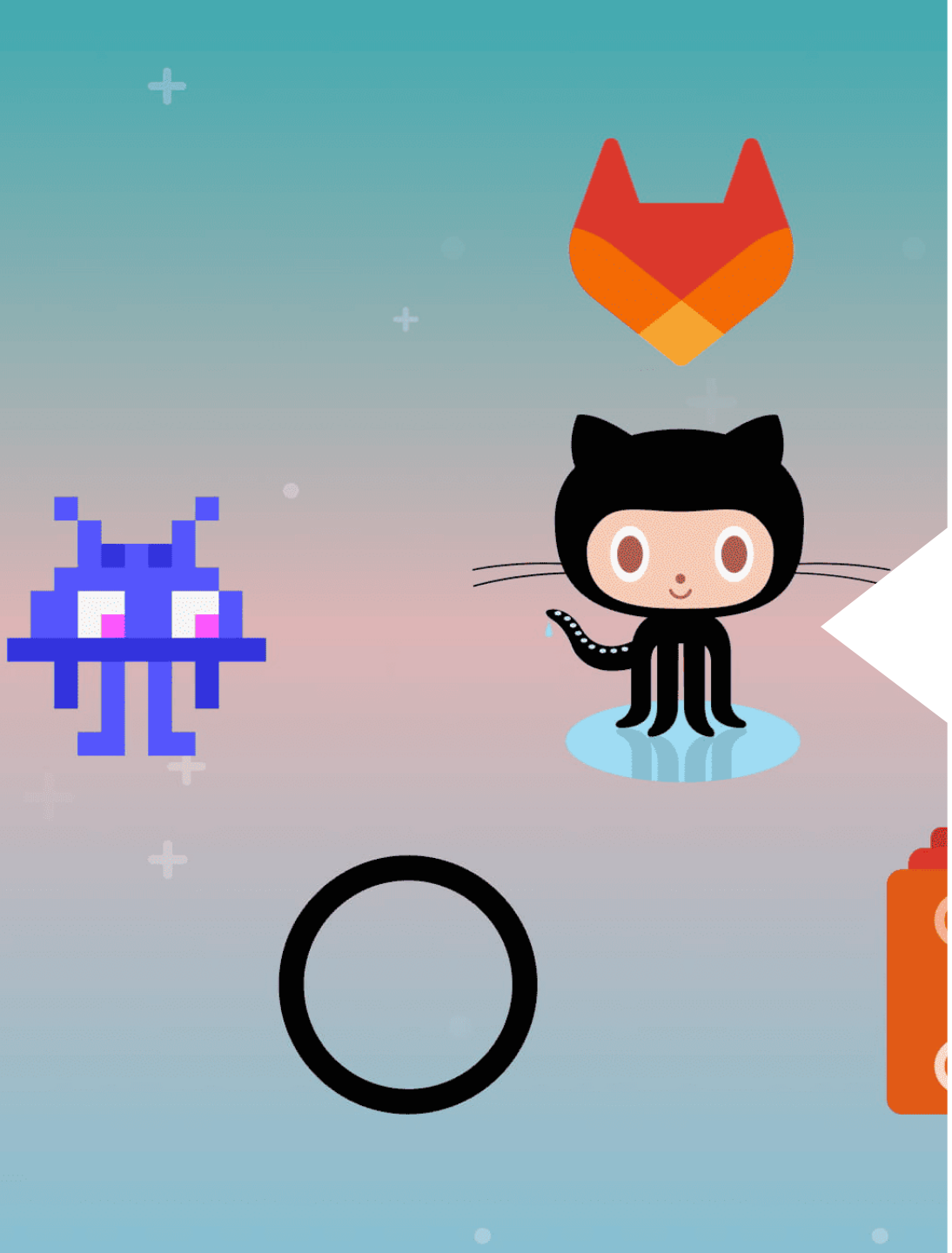
They're NOT the same thing!

GIT

- Software installed on your computer
- Tracks changes locally
- Works completely offline
- Free & open source
- Created in 2005 by Linus Torvalds (the person who built Linux)

GITHUB

- Website: cloud storage for your Git repos
- Stores your code online
- Enables collaboration with others
- Free for public repos
- Owned by Microsoft since 2018



GitHub Alternatives (FYI)

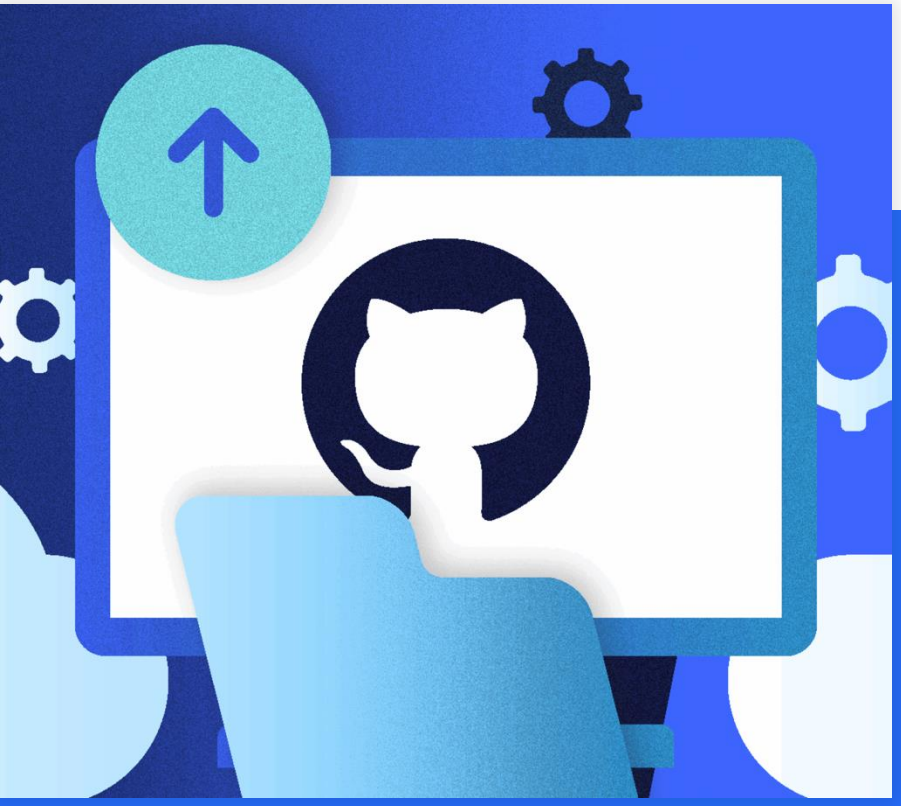
Other Options Exist:

- **GitLab** — Popular alternative, especially in enterprise companies
- **Bitbucket** — Owned by Atlassian, pairs well with Jira
- **SourceForge** — Old school, rarely used today

We're using GitHub because:

- Most popular — biggest community and job market presence
- Best platform for portfolios
- Free for public repos

Core Concept #1 – Repository



One repo = one project. Always.

Repository (Repo) — Your Project's Home Base

A project folder that Git tracks completely:

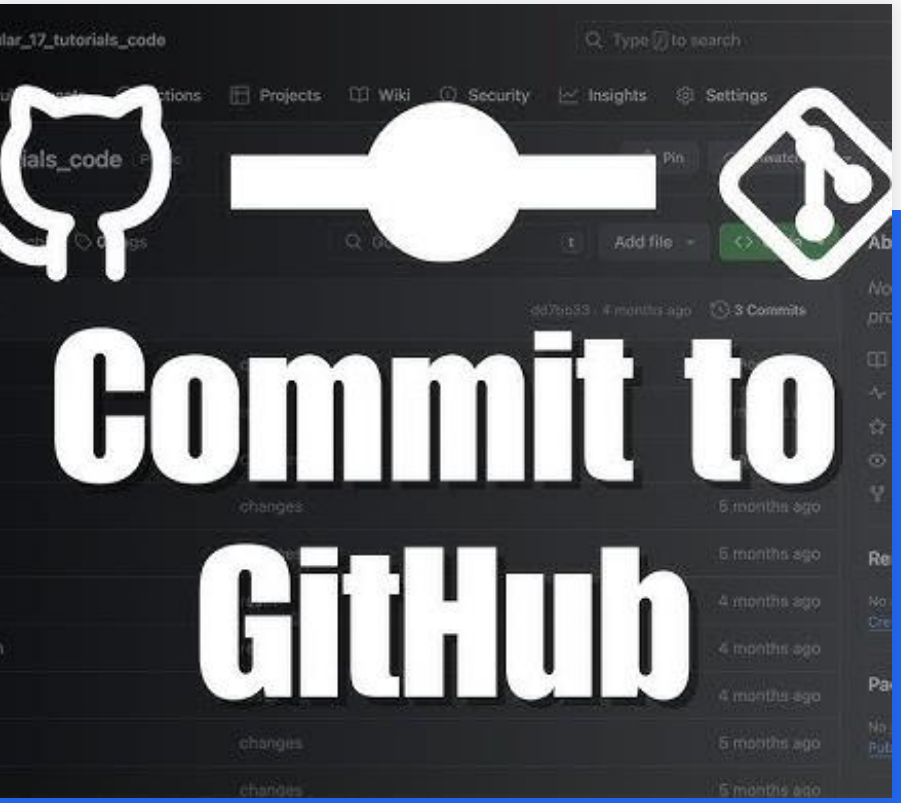
- Contains all your project files as usual
- PLUS the complete history of every change ever made
- PLUS who made each change and when
- PLUS the ability to jump back to any point in time

How to create one: two ways

- `git clone [url]`: download an existing repo from GitHub
- `git init`: turn any existing folder into a Git repo

Never delete the hidden .git folder inside your repo; that's where the entire history lives.

Core Concept #2 – Commit



Commit = Save Point in Time

Commit = Save Point in Time

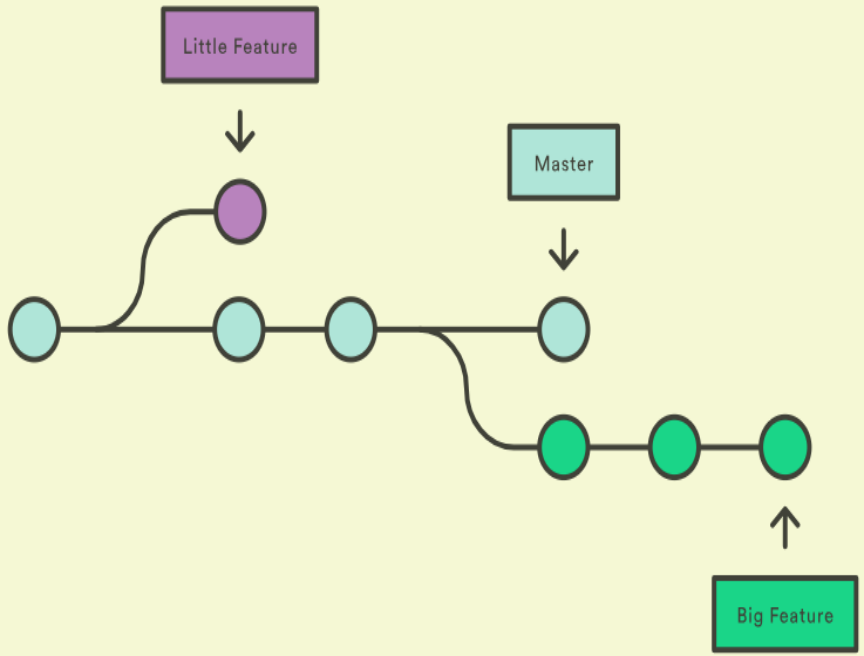
Like a save point in a video game; except each save remembers:

- A complete snapshot of your project
- Your message describing what changed and why
- Who made the change
- Exactly when it happened
- A unique ID called a **hash** (like `a3f2b1c`)

```
git commit -m  "Add homepage"        ← oldest
git commit -m  "Style navigation"
git commit -m  "Fix mobile layout"
git commit -m  "Add contact form"    ← newest
```

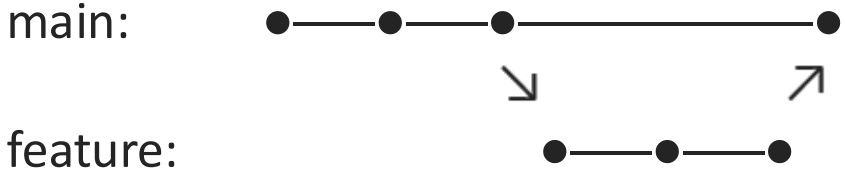
When should you commit? After completing any logical chunk of work — not at the end of the day, not once per file. Think: one task, one commit.

Core Concept #3 – Branch



Create a branch → experiment safely → merge back when ready

Branch = Alternate Timeline



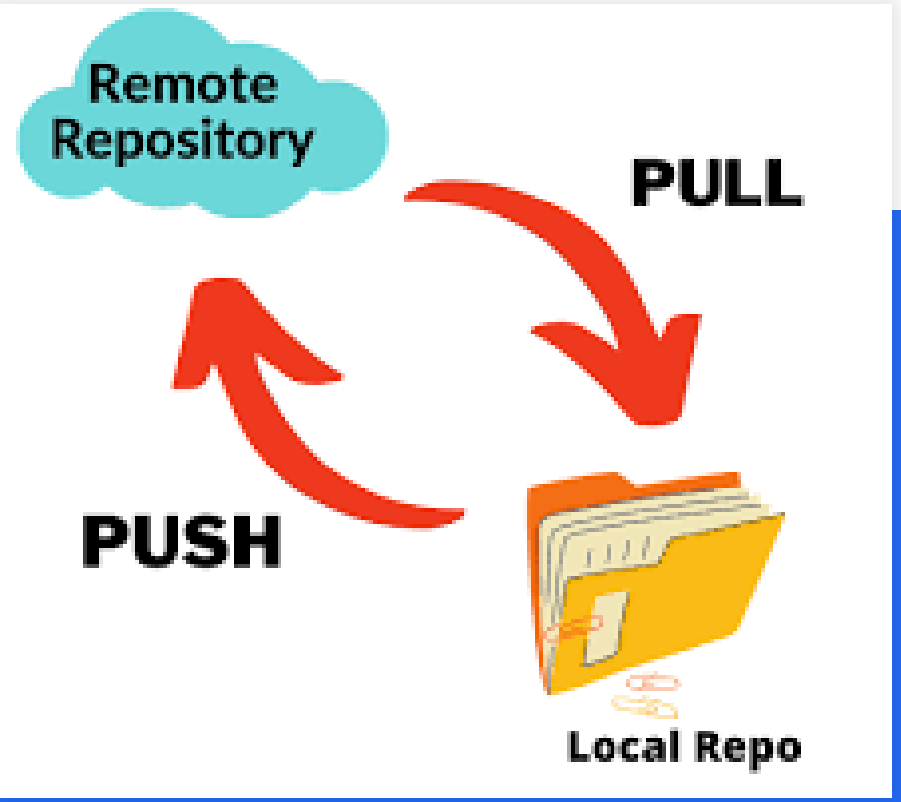
Why this matters:

- Work on new features without touching working code
- Experiment freely; main stays safe and stable
- Merge your changes back when they're ready

You're always working on a branch — right now it's called main.

Important: Older tutorials use "master" instead of "main" — they mean exactly the same thing.

Core Concept #4 – Remote



Clone once. Push and pull forever.

Remote = The Online Version of Your Repo

- **Local Repository:** on your computer
- **Remote Repository:** on GitHub

Each stage has one job:
•**Working Directory** — where you edit files
•**Staging Area** — where you choose what to save
•**Local Repo** — where commits live on your computer
•**Remote Repo** — where your work is safe and shareable on GitHub

The remote has a name it's called origin. When you see origin in commands or error messages, it just means GitHub.

The Git Workflow

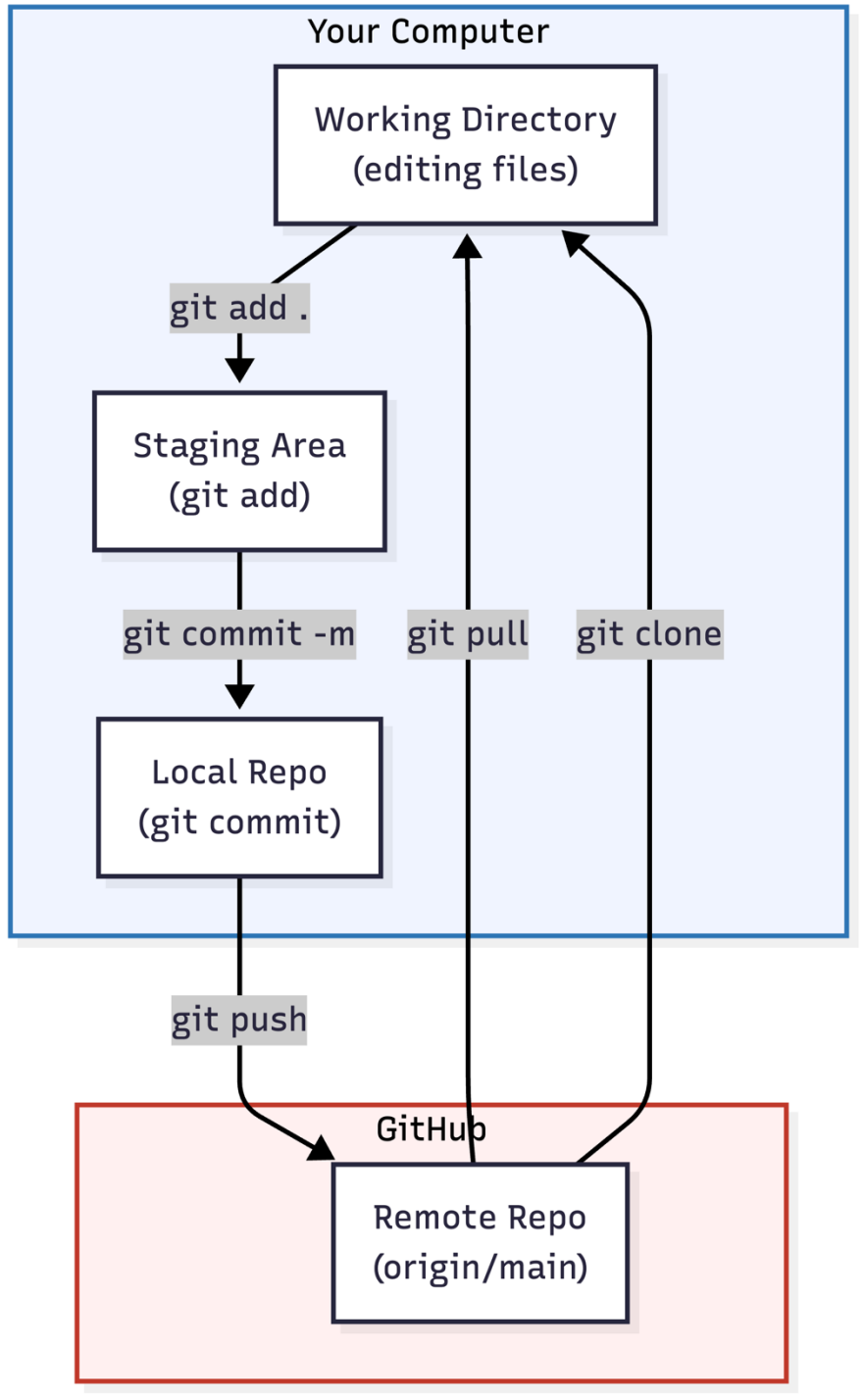
How Git Works — All Four Stages:

Each stage has one job:

1. **Working Directory:** where you edit files, write code, and make changes. Nothing is tracked yet.
2. **Staging Area:** where you choose exactly which changes to include in your next commit. Think of it as a loading dock.
3. **Local Repo:** where your commits are permanently saved on your computer. Works completely offline.
4. **Remote Repo:** where your work lives on GitHub, backed up, visible, and shareable with anyone.

The commands that move files between stages:

- `git add`: moves changes from Working Directory to Staging Area
- `git commit`: moves staged changes into your Local Repo
- `git push`: sends your Local Repo commits up to GitHub
- `git pull`: brings the latest GitHub changes down to your Working Directory
- `git clone`: downloads a remote repo to your computer for the very first time



Working Directory

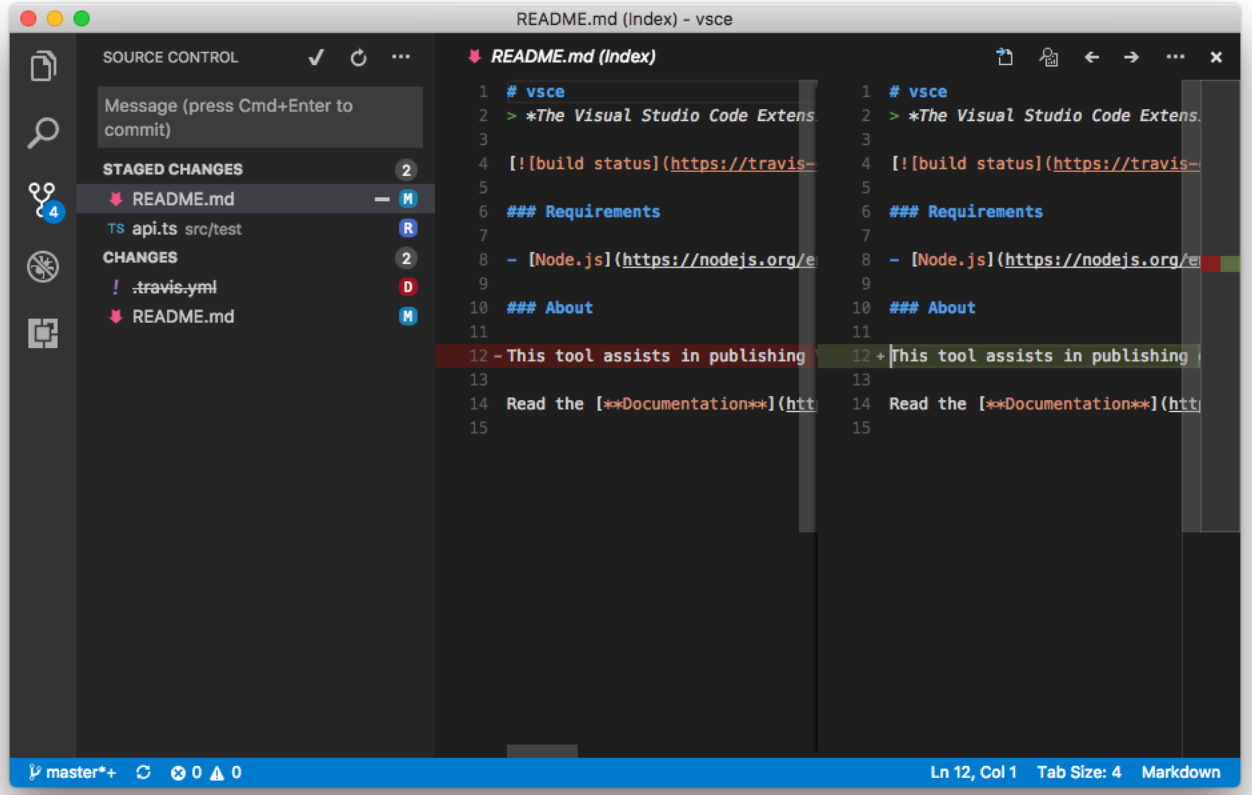
Stage 1: Working Directory

Where you edit, create, and delete files — just like you always have.

- Your project folder open in VS Code
- HTML, CSS, images — all your files live here
- Any change you make starts here
- Git is watching but hasn't recorded anything yet

VS Code shows your changes visually:

- **M** next to a file = Modified
- **U** next to a file = Untracked (new file Git hasn't seen before)
- The Source Control icon shows a badge count of how many files have changed



The working directory is just their project folder

Staging Area

Stage 2: Staging Area

Where you choose exactly which changes to include in your next commit.

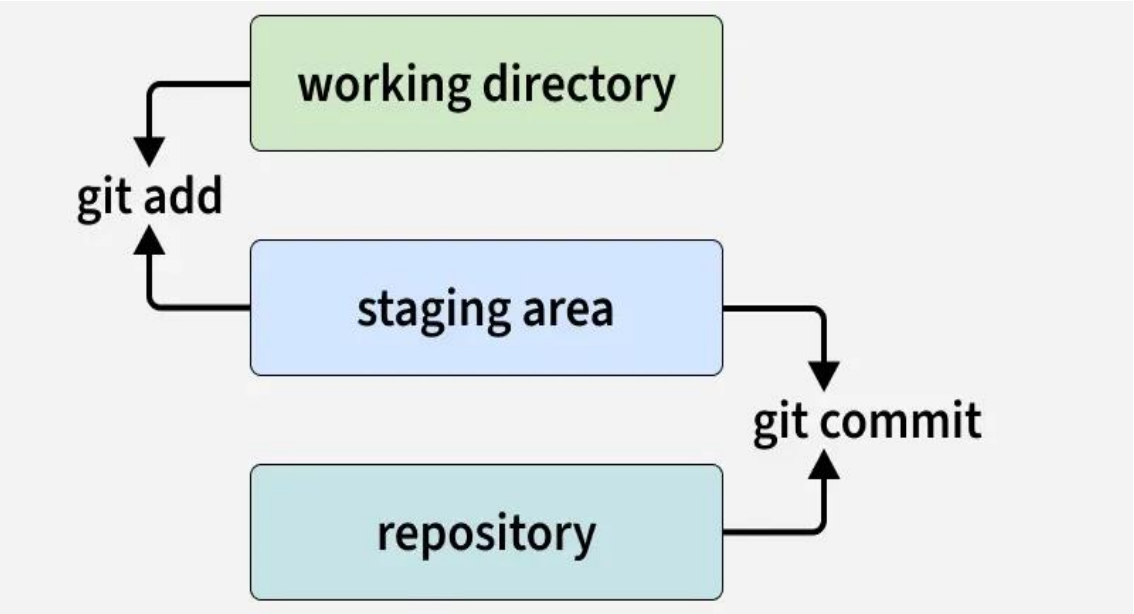
- `git add filename.html` — stage one specific file
- `git add css/style.css` — stage another specific file
- `git add .` — stage everything that changed

Why does staging even exist?

Imagine you spent the afternoon working and touched five files:

- Fixed a navigation bug in `header.html` ✓
- Updated colors in `style.css` ✓
- Started experimenting with a new layout — not ready yet ✗

Staging lets you commit just the finished work and leave the unfinished experiment out. Your commit stays clean and logical.



Always run `git status` after staging to confirm what's included.

Local Repository

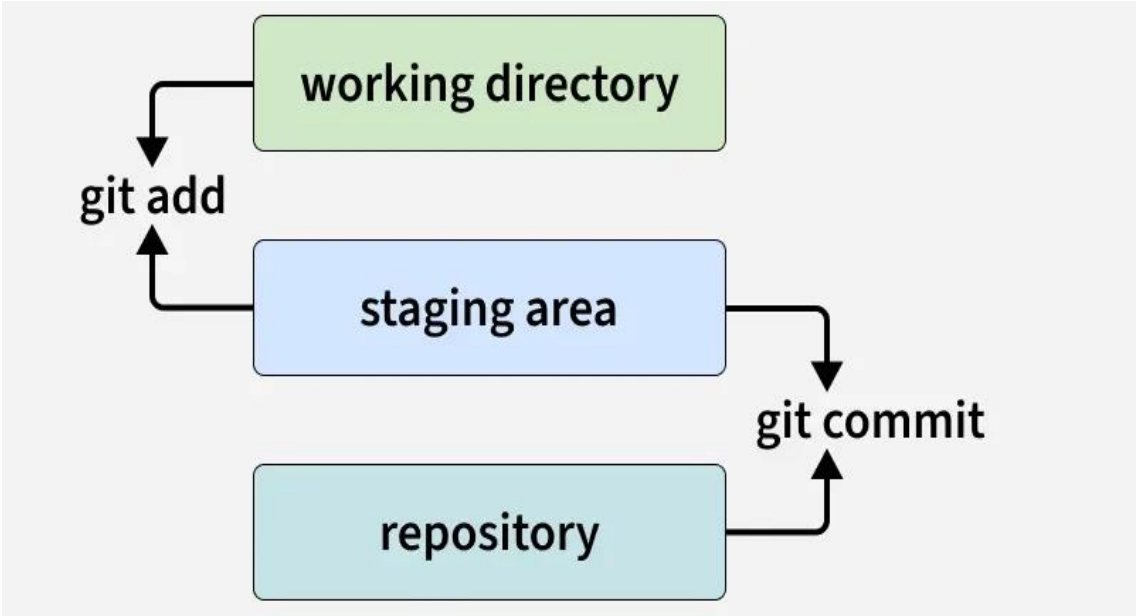
Stage 3: Local Repository

Where your committed snapshots are permanently stored — on your computer.

- `git commit -m "Add homepage”`: creates a new snapshot
- Each commit stores: your message, a timestamp, your name, and a unique hash ID
- Completely offline: no internet needed to commit
- History grows with every commit you make

Your commit history is a logbook of every decision you made on this project:

2b5a7f9 "Add contact form" Today, 3:42pm
f1c8d3b "Fix mobile layout" Today, 2:15pm
7d4e9a2 "Style navigation" Today, 1:08pm
a3f2b1c "Add homepage" Today, 11:30am



Commits live here first; before they go anywhere near GitHub.

Remote Repository

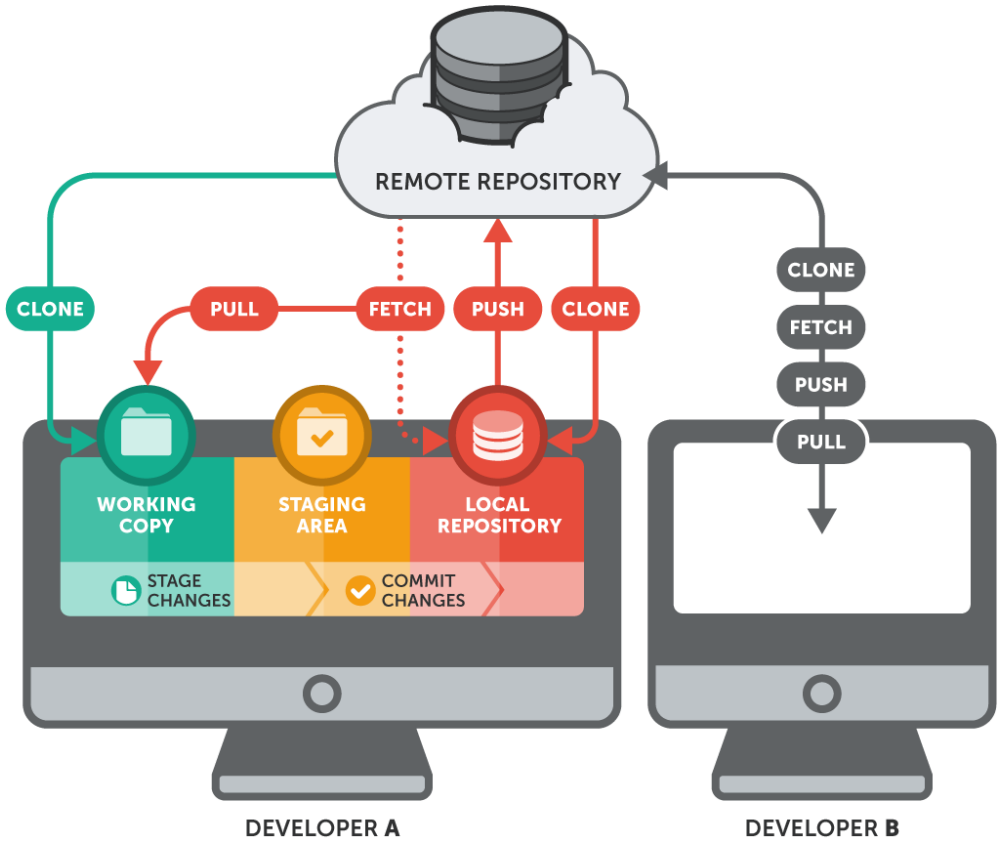
Stage 4: Remote Repository — Your Code on GitHub

Where your work is backed up, visible, and shareable with the world.

- `git push` — sends your local commits up to GitHub
- Accessible from any computer, anywhere, anytime
- Safe even if your laptop is lost, stolen, or broken
- Public repos are visible to anyone — including employers
- Each repo shows your files AND your full commit history

Your GitHub profile is more than a backup — it's a portfolio.
What employers see when they visit your GitHub:

- Your repos and what projects you've worked on
- How often you commit — shows consistency and work ethic
- The quality of your commit messages — shows professionalism
- How your code is structured — shows technical judgment



This is where your work becomes real and permanent beyond your own laptop.

The Basic Git Commands

90% of Git is Just These Commands:

Check what's happening:

- `git status` — see what files changed and what's staged

Save your work:

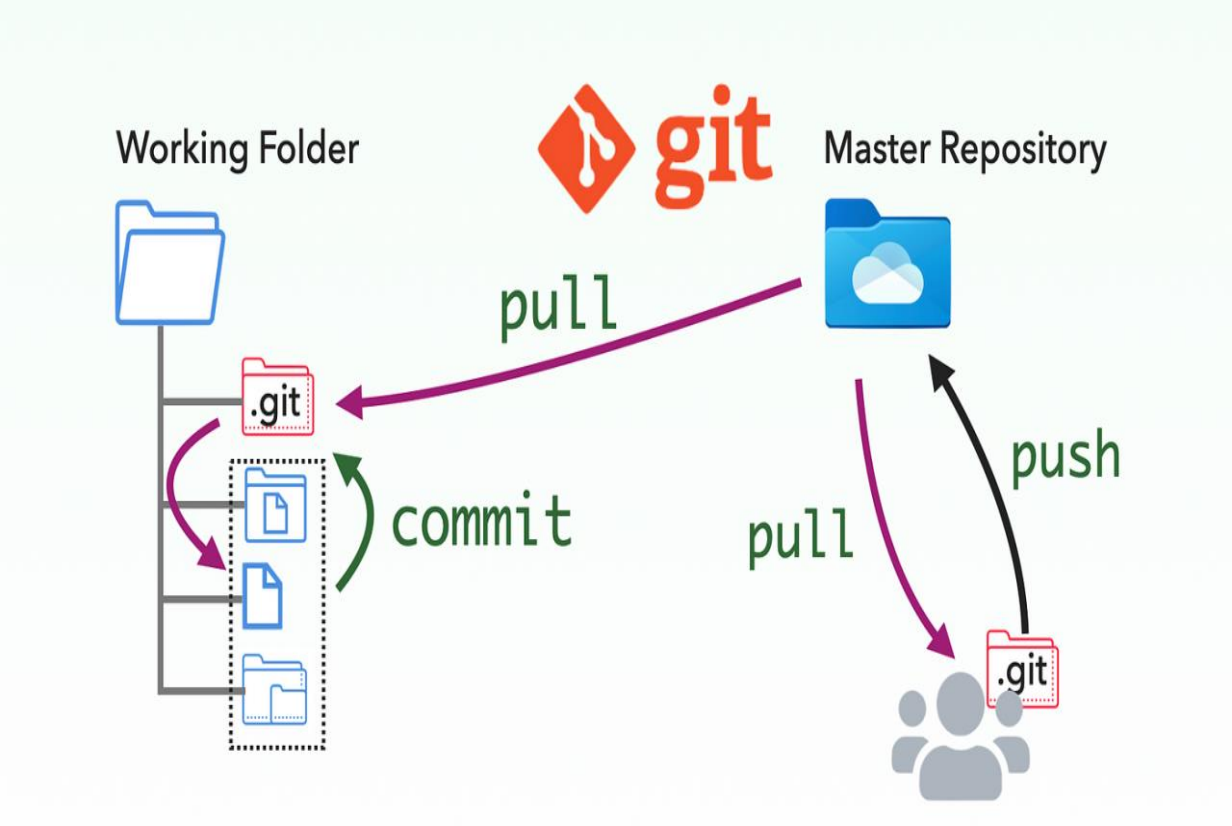
- `git add .` — stage all changes
- `git commit -m "message"` — create a snapshot

Sync with GitHub:

- `git push` — upload commits to GitHub
- `git pull` — download latest changes from GitHub

Review your history:

- `git log --oneline` — see all your commits in a compact list



That's it. Seriously. Six commands.

The order always matters: `status` → `add` → `commit` → `push`

Git Set Up



Set Up Git!



What We Need Before We Start:

- Git installed on your computer
- GitHub account created
- Git configured with your name and email
- GitHub authentication set up so pushing actually works

Where to get everything:

- Download Git: git-scm.com/downloads
- Create GitHub account: github.com
- GitHub Student Developer Pack: education.github.com/pack
- GitHub CLI (for authentication): cli.github.com
- Official setup guide: docs.github.com/en/get-started/quickstart/set-up-git

All four steps take less than ten minutes total.

Check if Git is Installed



First, Open Your Terminal:

- Mac: Press Cmd + Space, search "Terminal", press Enter
- Windows: Search "Git Bash" in the Start menu and open it

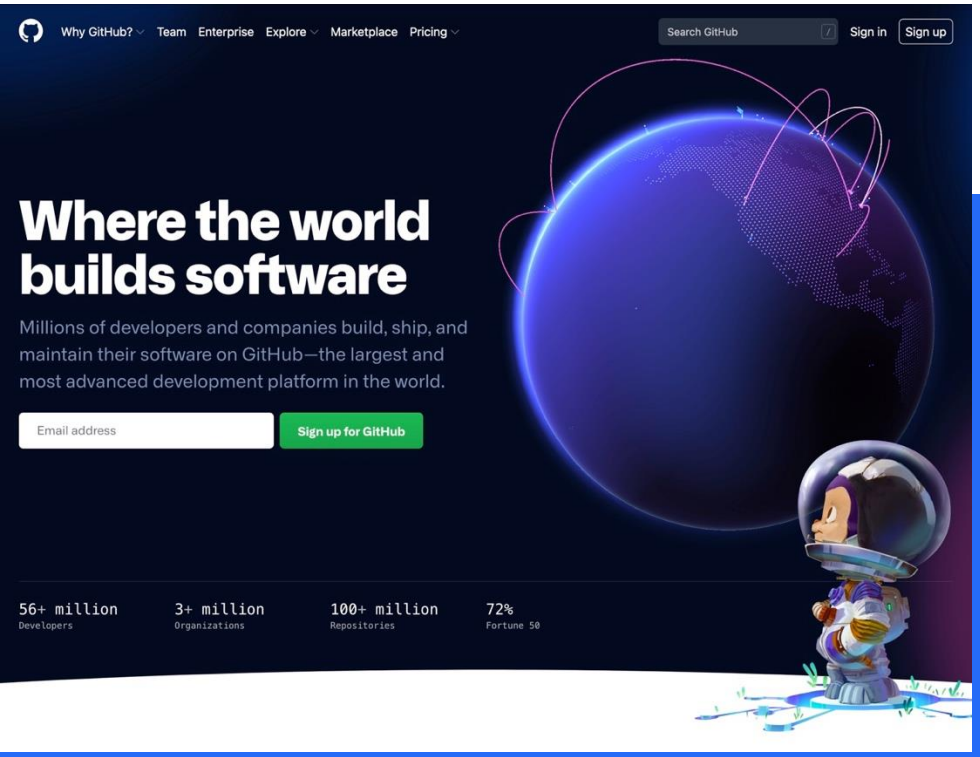
Windows users: use Git Bash for this class, not Command Prompt. Git Bash gives you the same commands as Mac Terminal, so all instructions work the same for everyone.

Now type this and press Enter:
git --version
You should see something like:
git version 2.50.1

If you see "command not found":

- Go to git-scm.com/downloads
- Download and install Git for your operating system
- Close and reopen your terminal
- Try the command again

Create GitHub Account



Let's Get You on GitHub:

1. Go to github.com
2. Click "Sign up"
3. Choose a professional username
4. Verify your email before doing anything else

Which email should you use? read this carefully:

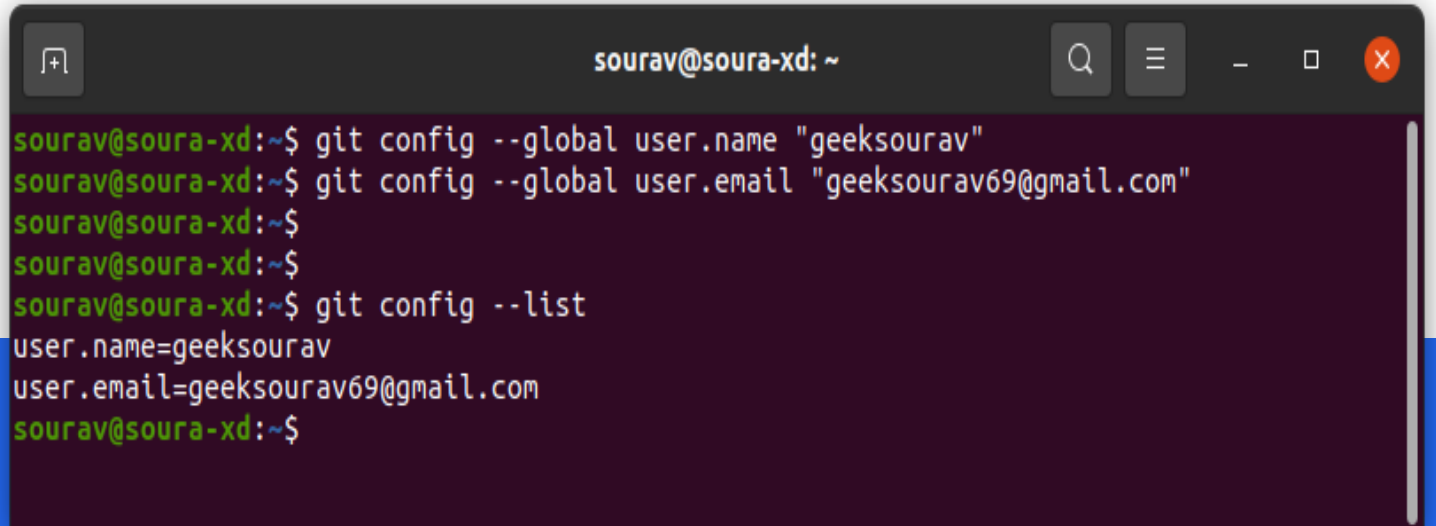
- Sign up with your **personal email** as your primary GitHub email
- Then add your **.edu email** as a secondary email in GitHub settings
- Apply for the Student Developer Pack using your .edu email at education.github.com/pack
- This way you keep full access to your account after graduation when your university deletes your .edu address

Why this matters: your GitHub account holds your entire portfolio and work history. If you sign up with only your .edu email and lose access to it after graduation, you could be locked out of your own account permanently.

Configure Git – One-Time Setup

Tell Git Who You Are:

1. This is a one-time setup. You never have to do it again on the same computer.
   ```
   git config --global user.name "Your Name”
   ```
2. This is the name that appears on every commit you make.
   ```
   git config --global user.email y
   ```
3. Use your personal email -- the same one you signed up to GitHub with.
   ```
   git config --global init.defaultBranch main
   ```
4. Makes sure your default branch is called "main", matching what GitHub uses. (Run this last to confirm everything looks correct before moving on)
   ```
   git config --list
   ```



Creating Your First Repository



Creating Your First Repository

Method 1: VS Code + Terminal (what we learned)

1. Open your project folder in VS Code
2. Open the terminal: Terminal menu, then New Terminal
3. Run these commands:

   ```
   git init
   git add .
   git commit -m "Initial commit"
   ```

4. Go to the Source Control panel in the left sidebar
5. Click "Publish Branch" -- VS Code connects to GitHub and creates the repo for you
6. Back in the terminal, run:

   ```
   git push
   ```

Method 2: GitHub first, then clone (common in tutorials)

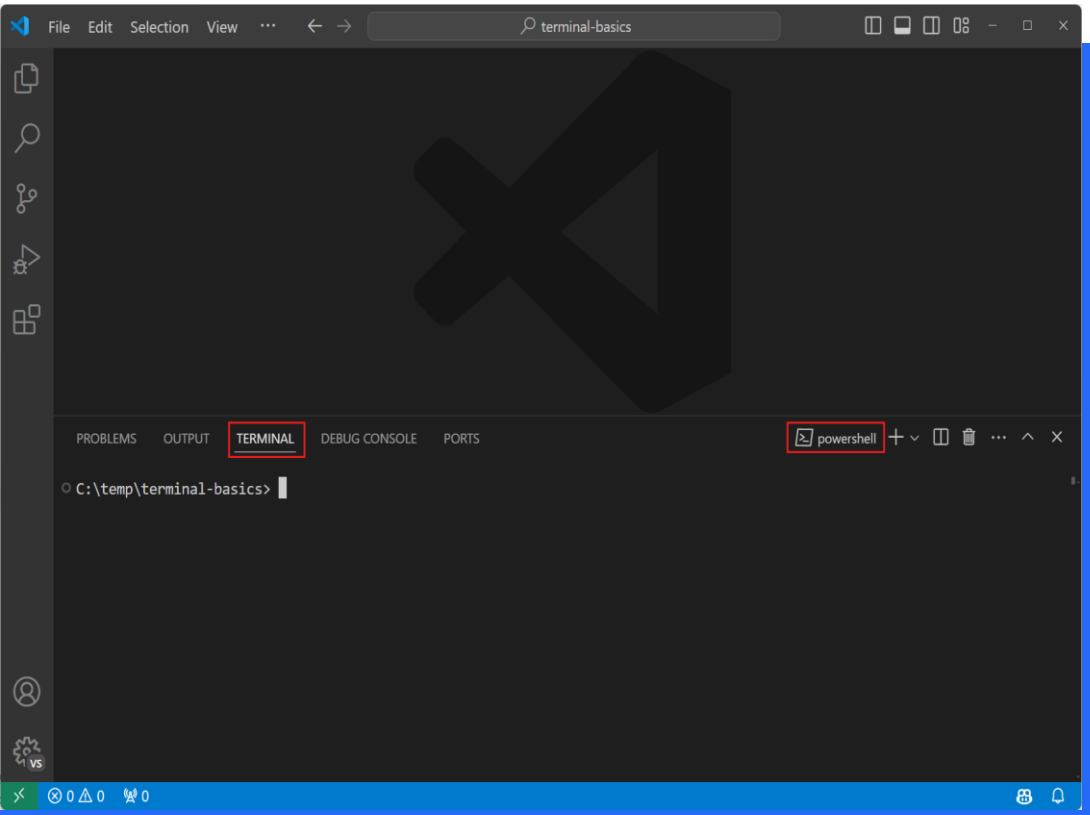
1. Create a new repo on github.com
2. Copy the repo URL
3. In VS Code terminal run:

   ```
   git clone [paste-your-url]
   ```

4. Open the cloned folder in VS Code
5. Start working -- the GitHub connection is already set up

Both methods get you to the same place. Method 1 is what we use in this class.
When you see Method 2 in tutorials outside of class, now you know what it is doing.

Step 1 - Open Your Project in VS Code



First, make sure you have a project folder ready. If you do not have one yet, create a new folder on your Desktop called web-design-practice.

Open it in VS Code:

- File menu, then Open Folder
- Navigate to your folder and select it

Then open the terminal:

- Terminal menu, then New Terminal
- The terminal opens at the bottom of VS Code
- It should already be pointing to your project folder

Confirm you are in the right place:

pwd

You should see the path ending in your folder name. If you do not, stop here and ask for help before continuing.

Step 2 - Initialize Git in Your Project



Tell Git to Start Tracking This Folder:

In your VS Code terminal, type:

git init

You should see:
Initialized empty Git repository in
/Users/yourname/Desktop/web-design-practice/.git/

That message confirms two things:

- Git is now watching this folder
- A hidden .git folder was created inside it -- this is where all your history will live

Look at your VS Code sidebar now:

- The Source Control icon should show a badge number
- Your files appear listed as untracked changes
- VS Code detected Git automatically

Step 3 - Create Your First File

Now Give Git Something to Track:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Git Project</title>
</head>
<body>
    <h1>Hello Git!</h1>
    <p>Learning version control.</p>
</body>
</html>
```

Now look at two things:

- The Source Control sidebar shows index.html with a **U** next to it (Untracked)
- The Explorer panel shows index.html with a **U** badge too

Git sees the file but is not tracking it yet.
That is exactly what we want at this stage.

Step 4 - Check What Git Sees

Before Doing Anything, Always Check Status First:

Using this command:

```
git status
```

You will see:

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present
```

Read this output carefully -- Git is telling you three things:

- You are on the main branch
- You have not made any commits yet
- index.html exists but Git is not tracking it yet

Color coding in the terminal:

- **Red** = file exists but is not staged yet
- **Green** = file is staged and ready to commit

Make git status a reflex. Run it before and after every single command.

Step 5 - Stage Your Changes

Tell Git Which Files to Include in Your Next Commit:

Using this command:

```
git add index.html
```

Or to stage everything at once:

```
git add .
```

Now run git status again -- always verify after staging:

```
git status
```

You will see:
On branch main

No commits yet

Changes to be committed:
(use "git rm --cached <file>..." to unstage)
new file: index.html

What just changed:

- index.html moved from red to green
- It is now listed under "Changes to be committed"
- VS Code Source Control panel moved it from "Changes" to "Staged Changes"

The file has not been saved to Git history yet; that happens in the next step.

Step 6 - Commit Your Changes

Create a Permanent Snapshot of Your Staged Files:

Using this command:

```
git commit -m "Add homepage"
```

You will see:

```
[main (root-commit) a3f2b1c] Add homepage
 1 file changed, 10 insertions(+)
 create mode 100644 index.html
```

Read that output -- Git is confirming three things:

- Which branch you committed to -- main
- Your commit message -- Add homepage
- What changed -- 1 file, 10 lines added

Writing good commit messages -- the professional standard:

- Use present tense: "Add homepage" not "Added homepage"
- Be specific: "Fix mobile navigation overlap" not "fix stuff"
- Keep it under 50 characters
- Complete this sentence: "If applied, this commit will..."

Step 7 - Publish to GitHub

Send Your Work to GitHub - Two Steps:

First time only - Publish your repo:
Go to the Source Control panel in the VS Code sidebar and click "Publish Branch"

VS Code will ask you two things:

- Public or Private - choose Public for class projects
- Repository name - keep it as your folder name

VS Code then does all of this automatically behind the scenes:

- Creates the repo on GitHub
- Connects your local repo to GitHub
- Pushes your first commit up

Every time after that - just push:

```
git push
```

Go check GitHub in your browser now:

- Visit github.com and open your repos
- You should see web-design-practice listed
- Click into it and see your index.html file
- Click "1 commit" to see your commit history

The Workflow From Here On

Now That Your Repo is on GitHub, This is Your Daily Loop:

Make a change to your file -- add a paragraph, update a heading, anything. Save the file with Cmd+S or Ctrl+S.

Then run these four commands:

git status

Confirm what changed.

git add .

Stage everything.

git commit -m "Add more content to homepage"

Save a snapshot.

git push

Then go check GitHub in your browser and refresh the page. You will see:

- Your updated file
- Your new commit listed in the history
- The commit message you wrote

This loop never changes. Every project. Every day.

Practice Time



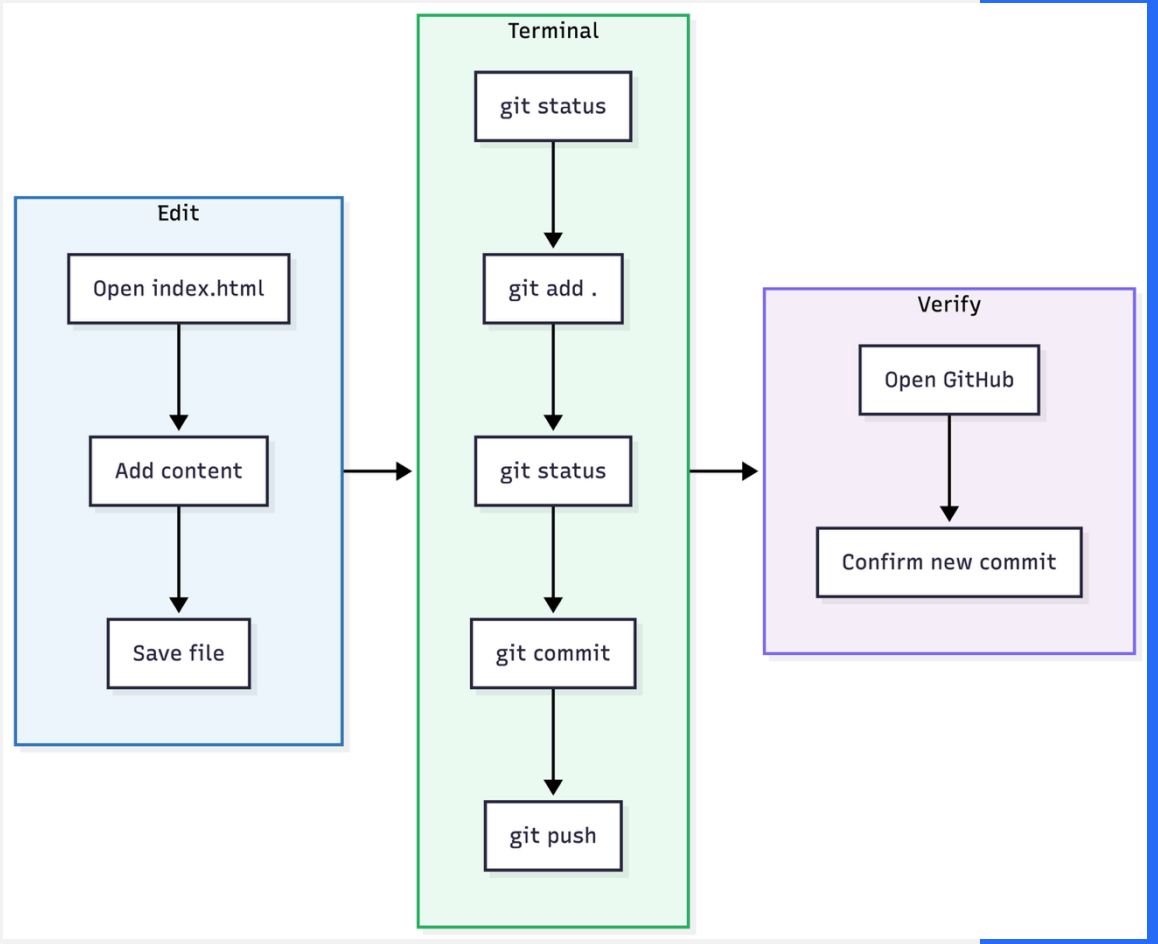
Practice Time - Let's Do It Again

Repeat the Loop on Your Own:

1. Open index.html in VS Code
2. Add something new: a second heading, a list, a paragraph
3. Save the file
4. Run through the full workflow:

   ```
   git status
   git add .
   git status
   git commit -m "Add more content to homepage"
   git push
   ```

5. Go to github.com and refresh your repo page
6. Confirm you can see your new commit in the history



Viewing Your History

See Everything You Have Done So Far:

You will see something like:

f2a3b1c Add more content to homepage
a3f2b1c Add homepage

Each line shows:

- The short hash - the unique ID for that commit
- Your commit message - the description you wrote

Three variations worth knowing:

Full details including author and date:

git log

Last 5 commits only:

git log --oneline -5

See exactly what changed in a specific commit:

git show a3f2b1c

Press **Q** to exit git log when you are done reading.



Undoing Changes - Git Has You Covered

Made a Mistake? Git Makes It Hard to Lose Work.

See what changed before staging:

git diff

Discard changes to a file - go back to the last commit:

git restore index.html

Unstage a file - keep the changes but remove from staging:

git restore --staged index.html

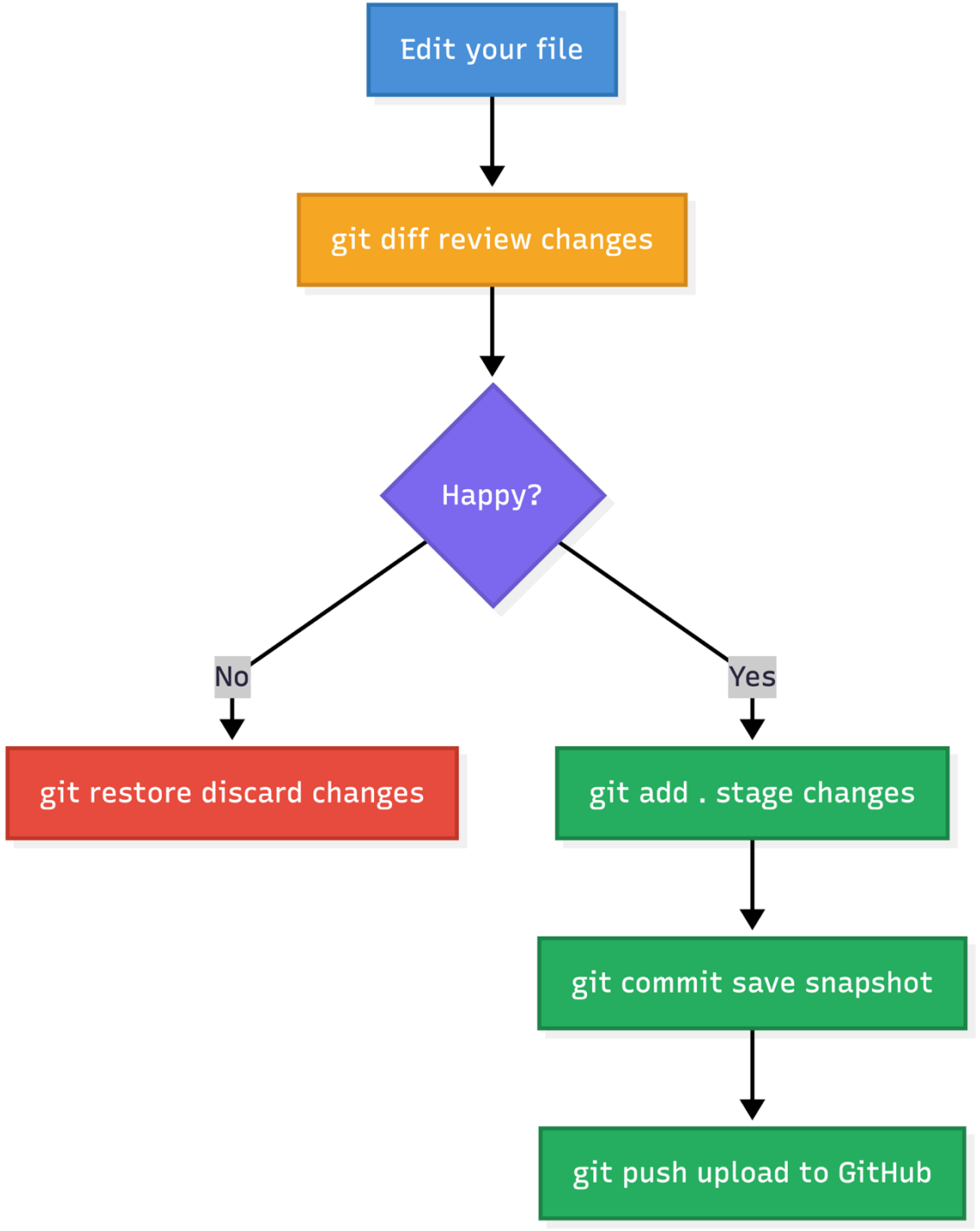
See a previous version of your project:

git checkout a3f2b1c

Then to come back to the latest version:

git checkout main

The most important rule: once you commit something, Git almost never loses it. Commit often and you will always have a way back.



Resources & Wrap up



Resources for Learning More

Official Reference:

- Git documentation: git-scm.com/doc
- GitHub documentation: docs.github.com
- Pro Git book - free, comprehensive, the definitive guide: git-scm.com/book

Interactive Practice:

- Learn Git Branching - the best visual Git tutorial online: learngitbranching.js.org
- GitHub Skills - free guided courses built by GitHub: skills.github.com

Student Benefits:

- GitHub Student Developer Pack: education.github.com/pack
- Apply tonight with your university email if you have not already

When You Get Stuck:

- Copy your exact error message and paste it into Google with quotes around it
- Stack Overflow has a solved answer for almost every Git error you will ever see

Git Command Cheat Sheet

One-Time Setup:

```
git config --global user.name "Your Name"
git config --global user.email "you@gmail.com"
git config --global init.defaultBranch main
```

Starting a Project:

```
git init                start tracking a folder
git clone [url]         download a repo from GitHub
```

Daily Workflow:

```
git status              see what changed
git add .               stage everything
git add filename.html   stage one file
git commit -m "message" save a snapshot
git push                upload to GitHub
git pull                download from GitHub
```

Reviewing History:

```
git log --oneline       see all commits
git log --oneline -5    see last 5 commits
git show HASH           see what changed in a commit
git diff                see unstaged changes
git diff --staged       see staged changes
```

Undoing Things:

```
git restore filename      discard changes to a file
git restore --staged file unstage a file
git checkout HASH         view an old version
git checkout main         return to latest
```

HAVE ANY QUESTIONS?

My push is not working, why?

Project 2?

I am not sure if my repo is on GitHub -- how do I check?

What happens if I forget to commit before pushing?

What is the difference between git add . and git add filename

Anything unclear?